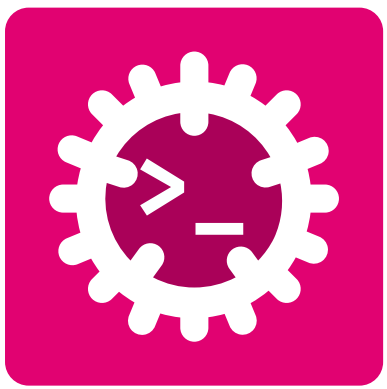


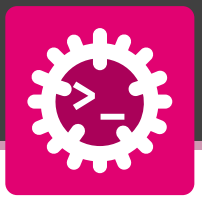
Advanced Programming

Concurrency // Parallel Programming

Silvan Zahno

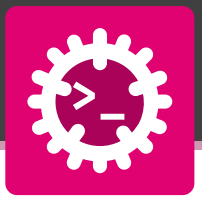
University of Applied Sciences Western Switzerland - HES-SO Valais Wallis
Systems Engineering
Infotronics

2026-08-11 - v0.1.0

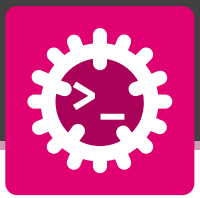


Contents

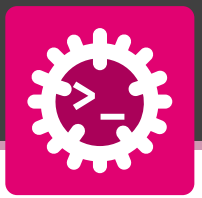
Contents



Contents



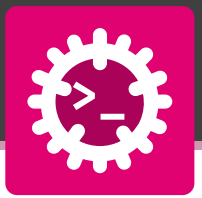
Contents



Concurrency

Doing more than one thing at once

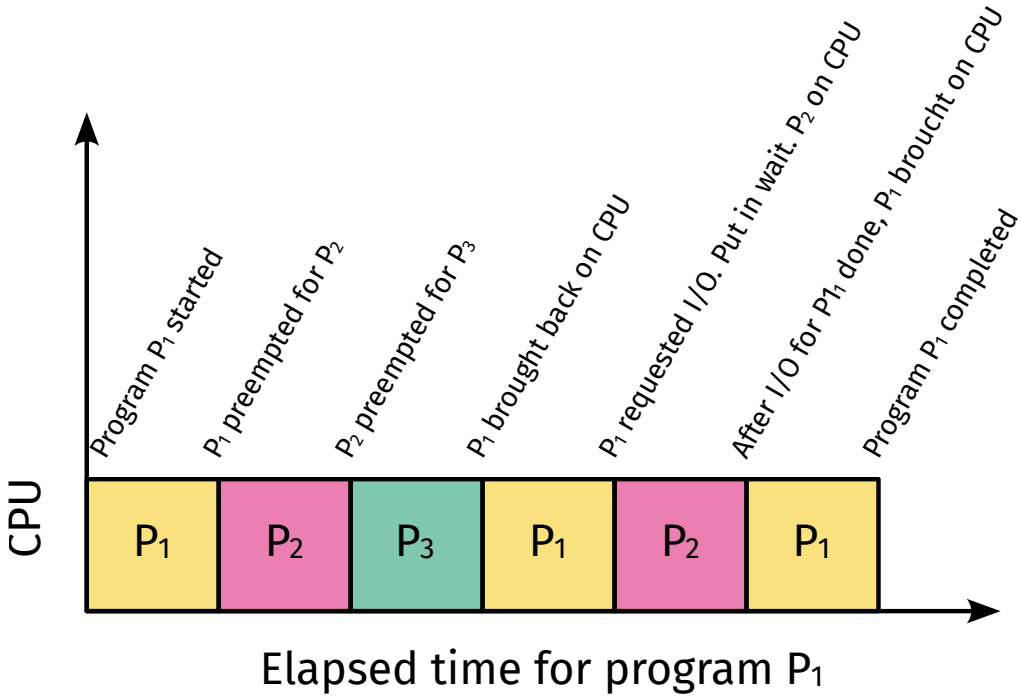
Why concurrency?



A program that does **one thing at a time** leaves the system underutilized.

- Programs spend a lot of time **waiting**
  - disk, network, the user
- That waiting time could be spent doing **other useful work**.

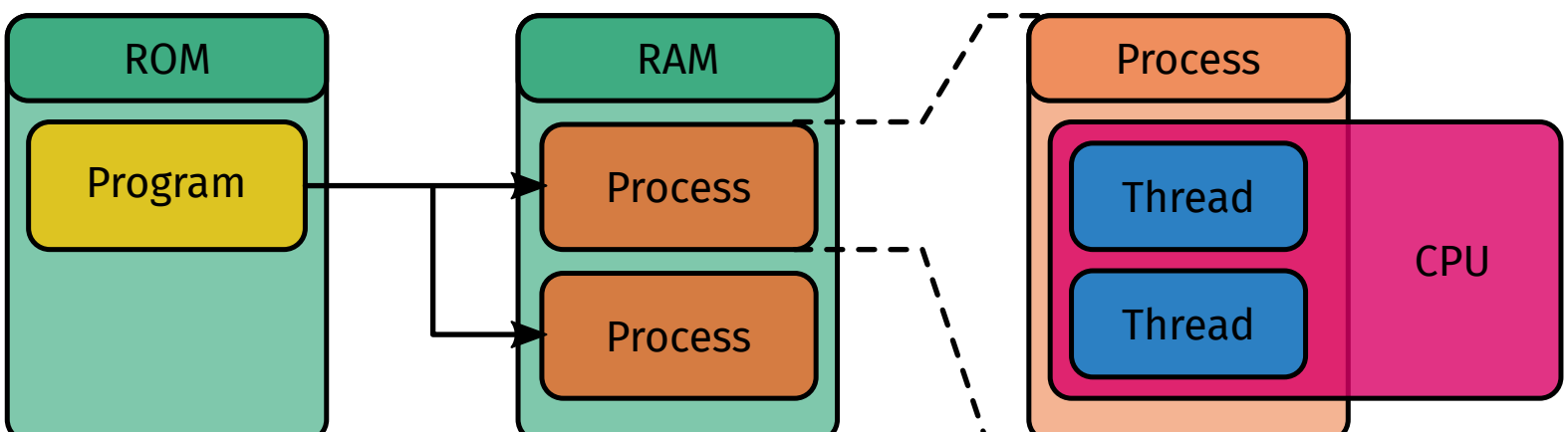
Goal: do more in the same wall-clock time, and stay **responsive**.



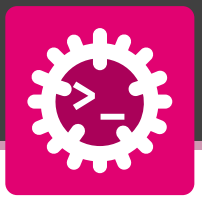
Program, Process, Thread

Three words we will use constantly - let's pin them down first.

- **Program** - code & data sitting **on disk** (not running)
- **Process** - a **running** instance of a program, with its **own memory**
- **Thread** - a **single path of execution** inside a process (the unit the OS **schedules**)

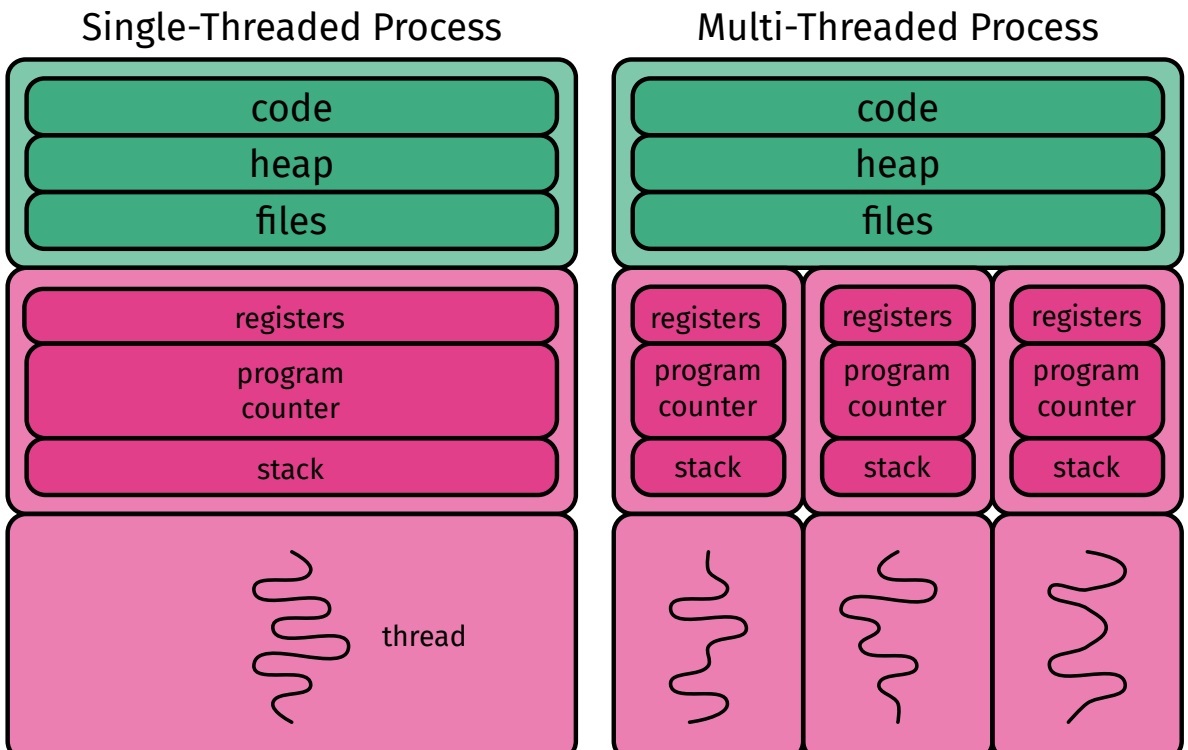


What threads share



A process can hold **several threads**. They **share** most of the process...

- **Shared** by all threads
  - **code**, **heap**, open **files**
- **Private** to each thread
  - its own **stack** and **registers**
- Sharing the heap is what makes threads **fast to communicate**...
- ...and exactly what makes them **dangerous**



The hazard: data races

A **data race** occurs when:

1. two or more threads access the **same** memory location,
2. at least one of these operation is a **write** access,
3. the operations are **unsynchronized** (no lock, no ordering guarentee between them)
4. the operations **overlap in time**, meaning they occur concurrently.

The result is **undefined** - corrupted data, random crashes.

Fearless concurrency

Rust rejects data races **at compile time**:

- **Ownership & borrowing**
  - one `&mut` **or** many `&`, never both
  - either **many readers**, or **one writer**, never both
- `Send` - a type is safe to **move** to another thread
- `Sync` - a type is safe to **share** (`&T`) between threads

If it **compiles**, it has **no data races**.

(Deadlocks & logic bugs 🐞 are still on you.)

Exercise - Concurrency Basics

Pick the **one** correct answer for each question.

1. What is private **to each thread?**

- a) the heap
- b) open files
- c) its stack and registers
- d) the program code

2. A data race needs all of these EXCEPT:

- a) two threads touch the same location
- b) at least one operation is a write
- c) the operations are guarded by a lock
- d) the operations overlap in time

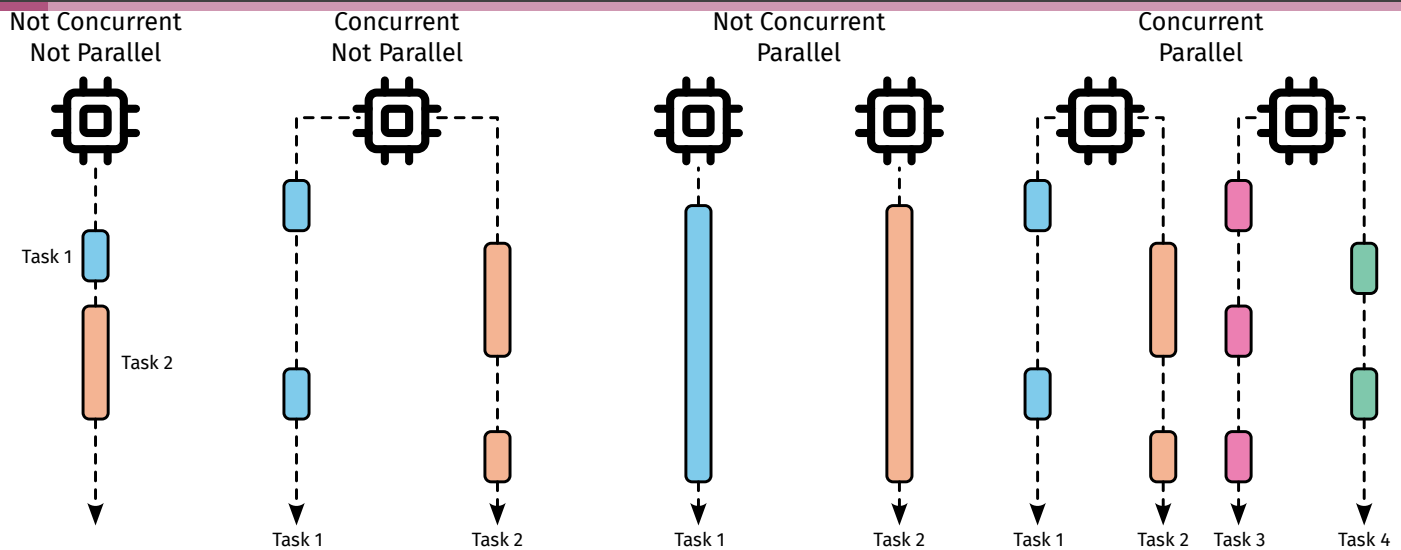
3. "If it compiles, no data races" - thanks to:

- a) a garbage collector
- b) ownership / borrowing + `Send` / `Sync`
- c) the OS scheduler
- d) running on a single core

Concurrency // Parallelism

Structure vs. simultaneous execution

Concurrency ≠ Parallelism



Concurrency - **structure**

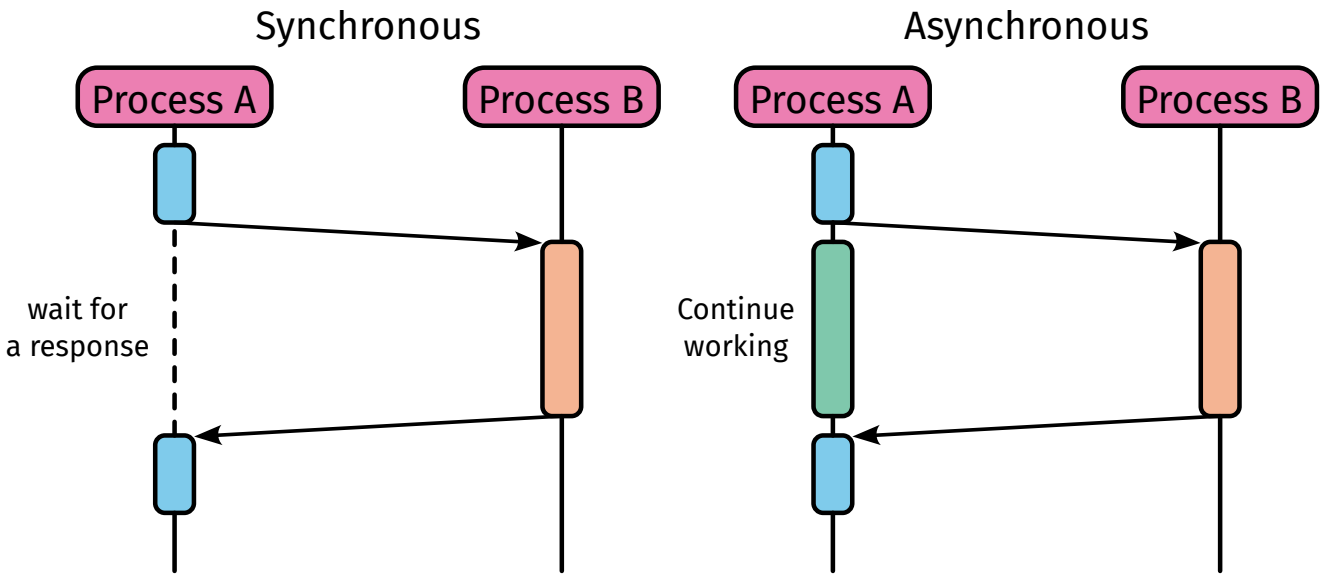
- many tasks that progress **independently**
- can **interleave** on a **single** core
- *"dealing with many things at once"*

Parallelism - **execution**

- tasks running at the **same instant**
- needs **multiple** cores
- *"doing many things at once"*

A program can be **concurrent** on one core, and becomes **parallel** when given more.

Synchronous vs. asynchronous



Synchronous

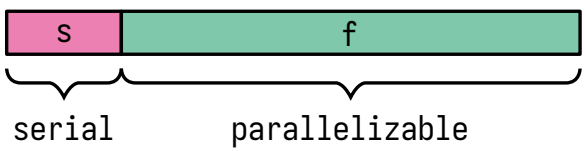
`Process B` must **wait** for `Process A`'s response before continuing.

Asynchronous

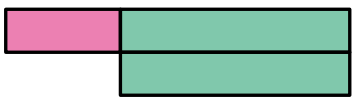
`Process A` **fires off** the request and **keeps working**, it handles the response later.

The same idea at every level: **don't block while you wait**.

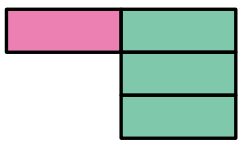
The limit: Amdahl's Law



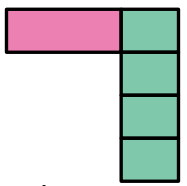
1 core



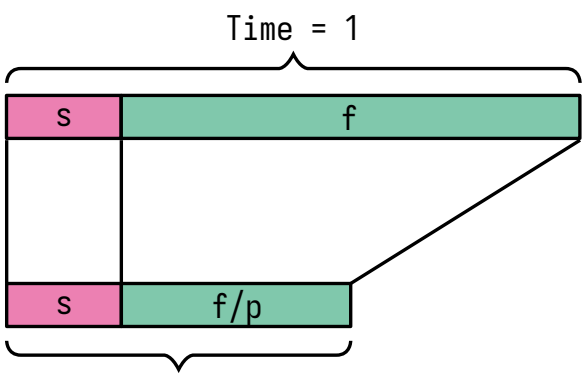
2 cores



3 cores



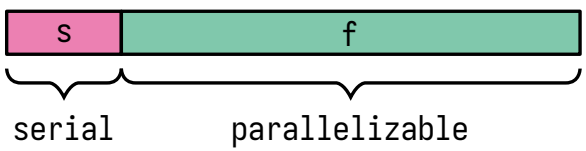
4 cores



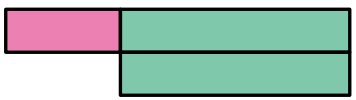
$$Time = s + \frac{f}{p}$$

If 50% of the execution time is sequential, the maximum speedup is 2x, no matter how many cores you throw at it.
- Gene Amdahl, 1967

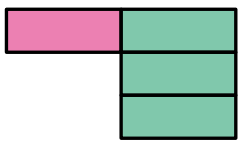
The limit: Amdahl's Law



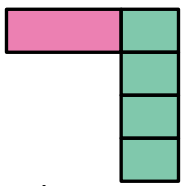
1 core



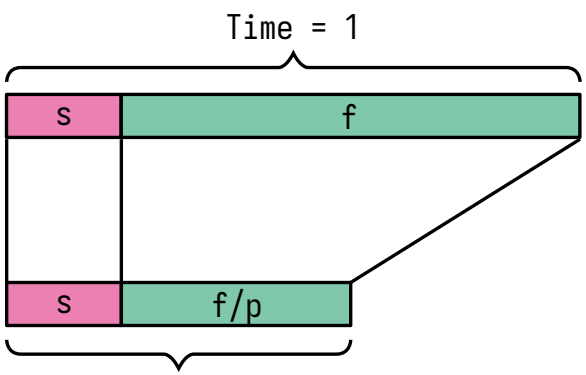
2 cores



3 cores



4 cores



$$Time = s + \frac{f}{p}$$

$$S = \frac{1}{s + \frac{f}{p}} = \frac{1}{1 - f + \frac{f}{p}}$$

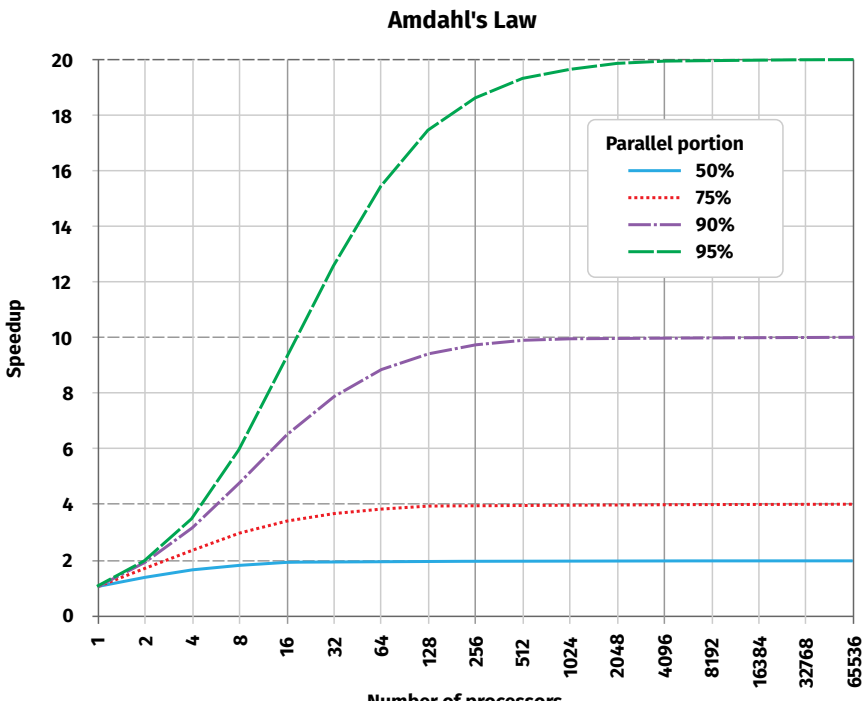
S = Speedup

$s = 1 - f$ = Serial fraction

$f = 1 - s$ = Parallel fraction

p = Number of processors

The limit: Amdahl's Law



- Every program has a **serial part** that **cannot** be parallelised.
- Only the **parallel part** benefits from more cores.
- Example: if **5%** is serial, the maximum speed-up is **20×**, ever.

> Adding cores has **diminishing returns**. Measure before you parallelise.

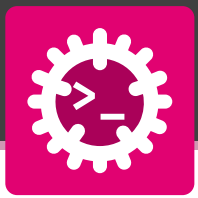
Exercise - Amdahl's Law

Floating point instructions improved to run 2x as fast, but only 10% of all executed instructions are Floating point. What will the overall speedup be?

Threads

Spawning and joining

Spawning a thread



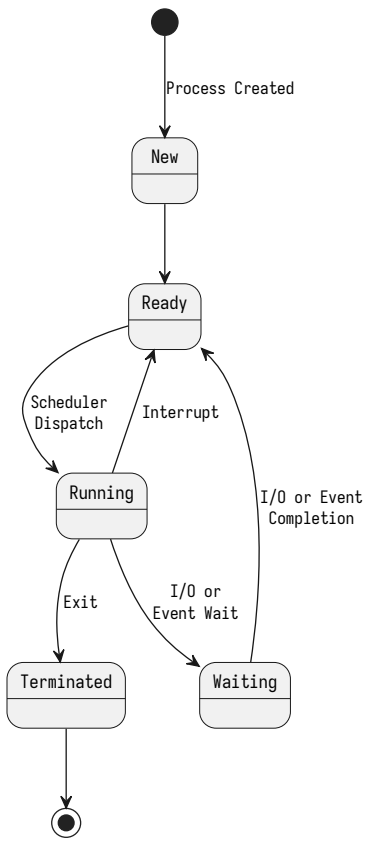
```rust
use std::thread;
use std::time::Duration;

fn main() {
  // spawn a thread that runs ALONGSIDE main
  let handle = thread::spawn(|| {
    for i in 1..=4 {
      println!("  thread: step {i}");
      thread::sleep(Duration::from_millis(100));
    }
    42 // value returned by the thread
  });

  // main keeps working at the SAME time
  for i in 1..=4 {
    println!("main: step {i}");
    thread::sleep(Duration::from_millis(100));
  }

  // join: wait for the worker, collect its result
  let result = handle.join().unwrap();
  println!("thread returned {result}");
}
```

```
main: step 1
  thread: step 1
main: step 2
  thread: step 2
main: step 3
  thread: step 3
main: step 4
  thread: step 4
thread returned 42
```

- `thread::spawn` starts a **new OS thread** that runs **alongside** `main`
- Both loops print **interleaved** - proof they run at the **same time**
- `join()` **blocks** until the thread finishes and **returns its value** (42)
- Without `join`, `main` could **end first** and kill the thread mid-run

Threads & the OS scheduler

A thread moves through a small **state machine** - the OS decides who runs when.



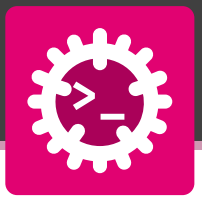
The states a thread is in:

- **Ready** - requires the Central Processing Unit (CPU), waiting its turn
- **Running** - currently executing on a core
- **Waiting** - blocked on Input/Output (I/O) or an event, not competing for the CPU

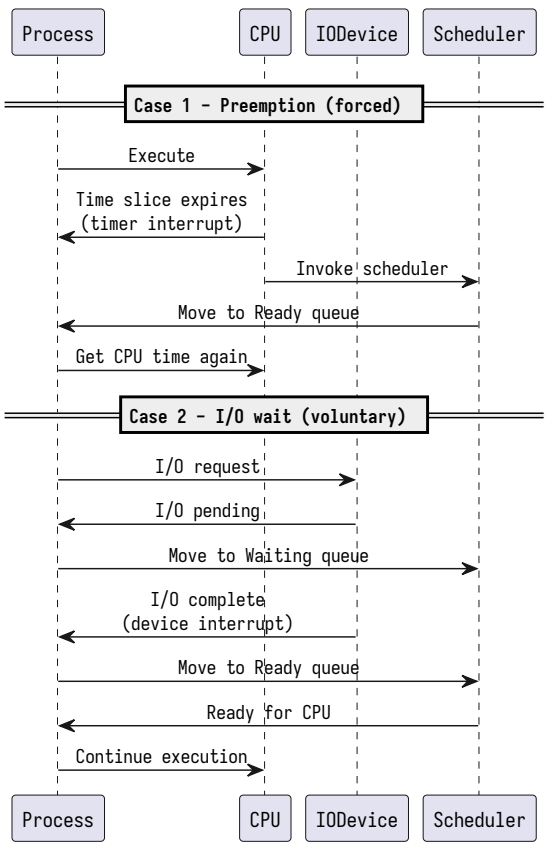
How it moves between them:

- **Dispatch** - the scheduler picks a Ready thread and runs it
- **Preemption** - the OS can **interrupt** a Running thread at any time, sending it back to Ready so another can run
- **Block / wake** - a thread that waits on I/O leaves the CPU when the I/O completes it returns to Ready

Preemptive scheduling in action



A running thread gives up the core in **two** ways - the scheduler then parks it and runs another.



Case 1 - preemption (forced):

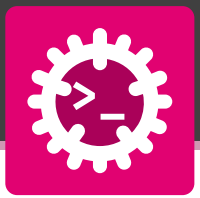
- The thread runs on the CPU until its **time slice** expires
- The CPU **invokes the scheduler**, which moves the thread to the **ready queue**
- It waits its turn, then gets the CPU back

Case 2 - I/O wait (voluntary):

- The thread issues an **I/O request** and can't continue
- It moves to the **waiting queue** - off the CPU, not competing for it
- When the device signals **completion** (an **interrupt**), the thread returns to the **ready queue**

> Either way the CPU is never idle - while one thread waits, another runs.

Data parallelism with Rayon (Rust crate)



```rust
use rayon::prelude::*;
use std::time::Instant;

fn cost(n: u64) -> u64 {
  (1..=n).map(|i| i % 7).sum()
}

fn main() {
  let nums: Vec<u64> = (1..=20_000).collect();

  // ── sequential ──
  let start = Instant::now();
  let _seq: u64 = nums.iter().map(|&n| cost(n)).sum();
  let seq_time = start.elapsed();

  // ── parallel (Rayon) ──
  let start = Instant::now();
  let _par: u64 = nums.par_iter().map(|&n| cost(n)).sum();
  let par_time = start.elapsed();
  let speedup = seq_time.as_secs_f64() / par_time.as_secs_f64();
  println!("sequential: {seq_time:?}");
  println!("parallel:   {par_time:?}");
  println!("speedup:    {:.2}x", speedup);
}
```

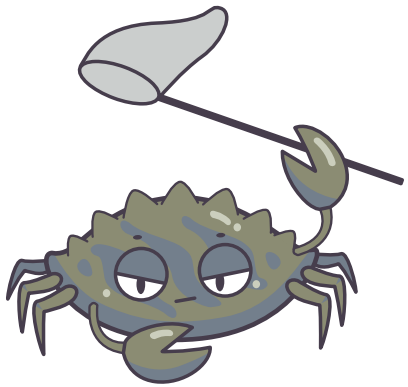
```
# Macbook Pro M4 Max (16cores)
sequential: 1.553656209s
parallel:   133.673667ms
speedup:    11.62x
```

- Compare `iter()` (sequential) vs `par_iter()` (parallel), timed with `Instant`
  - One line changes - `iter()` ⇨ `par_iter()` - and Rayon splits the work over a **thread pool** (≈ one thread per core)
  - **11.6× faster** here on a CPU-bound, **independent** workload
  - Rust's safety guarantees still hold - **no data races**, enforced at compile time

Exercise - Threads

Read the program and give the value it prints. Does the **thread order** change the result?

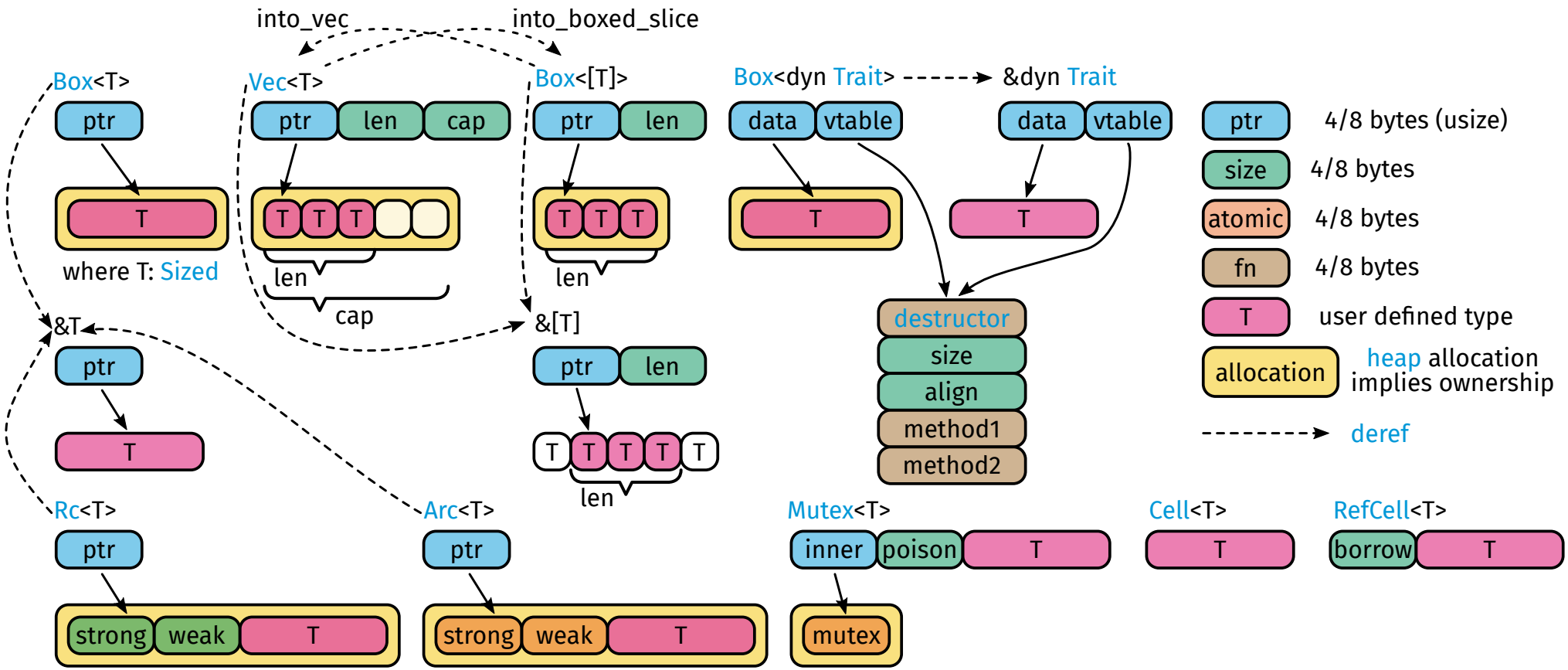
```
use std::thread;

fn main() {
  let mut handles = vec![];
  for i in 1..=3 {
    handles.push(thread::spawn(move || i * 10));
  }

  let mut total = 0;
  for h in handles {
    total += h.join().unwrap();
  }
  println!("total = {total}"); // ?
}
```



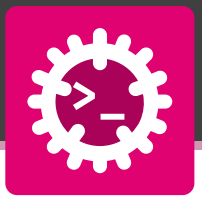
Smart Pointers

Owning and sharing values safely

Smart Pointers Overview



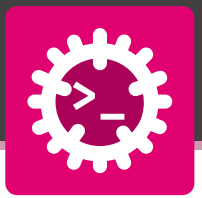
Smart Pointers Overview



A **smart pointer** is a type that owns data **and adds behaviour** (heap allocation, reference counting, locking, runtime borrow checks). A plain `&T` is just a **borrow** - not a smart pointer.

Type	Group	Description
Single owner	`Box<T>`	Puts a value on the **heap** instead of the stack, with **one** owner. Use it for data too big for the stack, or for **recursive** types (a struct that contains itself) where the size would otherwise be infinite.
	`Box<dyn Trait>`	A `Box` holding a **trait object**: "some type implementing `Trait`, decided at run time." Lets you own a value when you don't know its concrete type - e.g. one element in a list of mixed shapes.
Shared owner	`Rc<T>`	**Reference-counted** shared ownership: many parts of your program co-own one value. It counts the owners and frees the value when the **last** one is dropped. **Single thread only** (the count isn't thread-safe).
	`Arc<T>`	The same as `Rc`, but the owner count mutations are **atomic**, so it is safe to share **across threads**. Slightly slower than `Rc`; use it only when threads are involved.

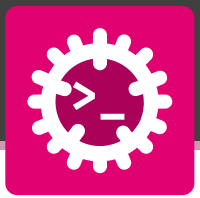
Smart Pointers Overview



Type	Group	Description
Interior mutability	`Cell<T>`	Lets you **change** a value even through a shared `&` reference, by swapping the whole value in and out with `get` / `set`. For small `Copy` types (numbers, flags). No borrowing, never panics.
	`RefCell<T>`	Like `Cell`, but lets you **borrow** the inner value (`borrow` / `borrow_mut`). The borrow rules (one writer or many readers) are checked **while the program runs** instead of at compile time - and it **panics** if you break them.
	`Mutex<T>`	Guards data so **one thread at a time** can touch it. `lock()` blocks until it's your turn, hands you exclusive access, and **unlocks automatically** when you're done. The way to mutate shared data across threads.
	`RwLock<T>`	Like `Mutex`, but smarter about reading: it allows **many readers at once** OR **one writer**. Use it when reads are frequent and writes are rare.

Interior mutability lets you mutate data behind an immutable reference by moving borrow checking from compile time to runtime.

Choosing a smart pointer

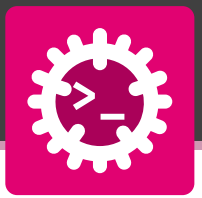


Ask yourself two questions: **How many threads?** and **read or write?**

Need	**Single thread**	**Multi-thread**
heap, **one owner**	`Box<T>`	`Box<T>`
shared ownership (read)	`Rc<T>`	`Arc<T>`
shared + mutable	`Rc<RefCell<T>>`	`Arc<Mutex<T>>` `Arc<RwLock<T>>`
interior mutability mutate a `Copy` field via `&self`	`Cell<T>`	`Atomic*`

- The columns **mirror** each other: `Rc` ⇔ `Arc`, `RefCell` ⇔ `Mutex`, `Cell` ⇔ `Atomic`
- Same three ideas - **own**, **share**, **mutate-through-shared** - each with a **thread-safe** twin
- **Rule**: pick the **single-thread** tool unless you actually cross threads (it's cheaper)

Heap, one owner: Box<T>



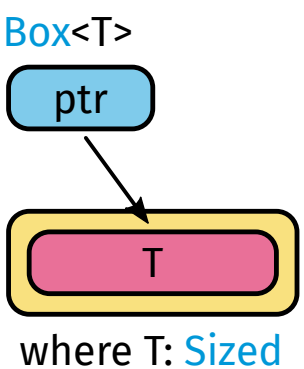
Concept: put **one** value on the **heap**.

- The **same** tool single- or multi-threaded (move it across with `move`).
- The value is freed when the `Box` drops (RAII).

```rust
use std::thread;

// 4 KB grid: too big for the stack ⇒ heap
let grid = Box::new([[0u8; 64]; 64]);

// `move` transfers the ONE owner to the worker
let h = thread::spawn(move || grid.len());
println!("rows: {}", h.join().unwrap()); // 64
```

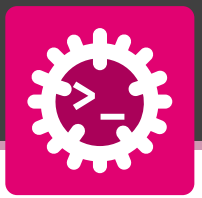
Box<T>

- one value on the **heap**
  (data too big for the stack, or a **recursive** type)
- `move` hands that **sole owner** to a thread



where T: Sized

Heap, one owner: `Box<dyn Trait>`



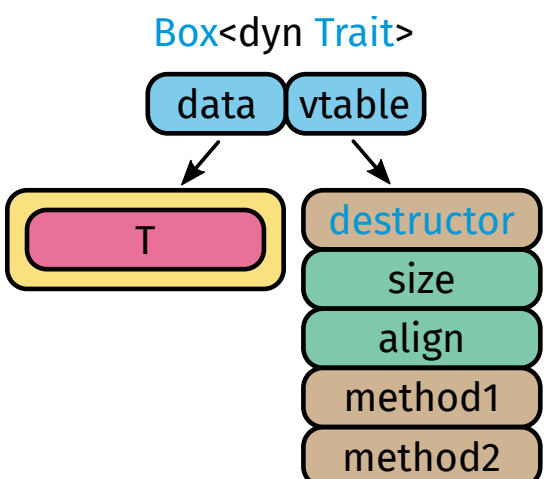
Concept: put **one** value on the **heap**.

- The **same** tool single- or multi-threaded (move it across with `move`).
- The value is freed when the `Box` drops (RAII).

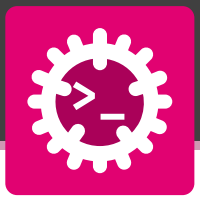
```
trait Job: Send { fn run(&self); }
struct Email;
impl Job for Email {
    fn run(&self) { println!("sending"); }
}
// concrete type hidden behind the Box
let job: Box<dyn Job> = Box::new(Email);
// move the trait object onto a worker thread
thread::spawn(move || job.run()).join().unwrap();
```

`Box<dyn Trait>`

- owns **some** type implementing `Trait`, picked at **run time** (dynamic dispatch).
- A `Send` object can move across threads.



Shared ownership (single thread): Rc<T>



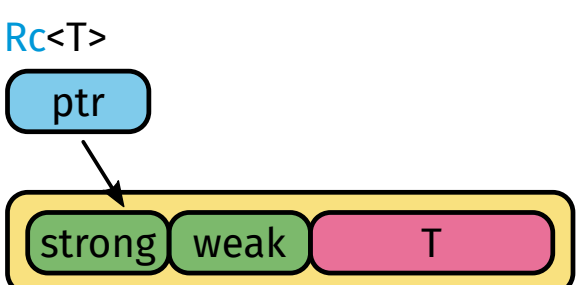
Concept: many owners of **one** read-only value

- a non-atomic **reference count** frees it when the **last** owner drops.

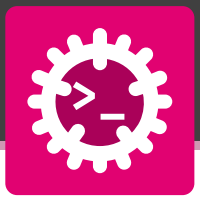
```rust
use std::rc::Rc;

struct Config { db_url: String, retries: u32 }
struct Service { cfg: Rc<Config> }

fn main() {
    // build the read-only config ONCE
    let cfg = Rc::new(Config {
        db_url: "postgres://db:5432".into(),
        retries: 3,
    });

    // every service co-owns the SAME config, no data copy
    let api    = Service { cfg: Rc::clone(&cfg) };
    let worker = Service { cfg: Rc::clone(&cfg) };

    println!("owners: {}", Rc::strong_count(&cfg)); // 3
    println!("{} / {}", api.cfg.db_url, worker.cfg.retries);
}
```

Rc<T>

- many owners of one **immutable** value
- `clone` bumps a **non-atomic** count
  ⇨ **no deep copy**
  - last owner dropped ⇨ value **freed**
- **single thread only** (count isn't thread-safe)



Shared ownership (multi-thread): Arc<T>



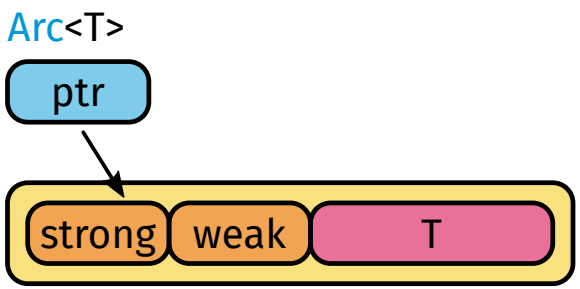
Concept: many owners of **one** read-only value, but the count is **atomic** ⇨ safe to share **across threads**

- Slightly slower, use it only when threads are involved.

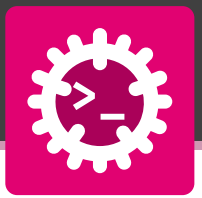
```rust
use std::sync::Arc;
use std::thread;

fn main() {
    // load a big read-only dataset ONCE
    let data = Arc::new(vec![7, 2, 9, 4, 8, 1, 5, 3]);

    let mut handles = vec![];
    for t in 0..4 {
        let data = Arc::clone(&data);    // owner per thread
        handles.push(thread::spawn(move || {
            // each thread reads its own slice, no copy
            data[t * 2..t * 2 + 2].iter().sum::<i32>()
        }));
    }

    let total: i32 = handles.into_iter()
        .map(|h| h.join().unwrap()).sum();
    println!("total = {total}");         // 39
}
```

Arc<T>

- **atomic** reference count ⇨ Send + Sync
- share **immutable** data across threads, **no copy**
- each thread gets its own **owner** via `clone`
- need to **mutate**? add a lock
  ⇨ `Arc<Mutex<T>>`



Rc/Arc: strong vs. weak counts



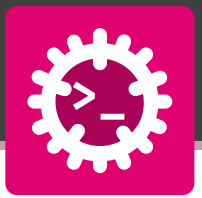
Every Rc/Arc carries **two** counts. The value lives as long as **strong > 0**.

```rust
use std::rc::{Rc, Weak};

let a = Rc::new(5);
let b = Rc::clone(&a);                  // strong owner
let w: Weak<i32> = Rc::downgrade(&a); // weak, non-owning

println!("{}", Rc::strong_count(&a)); // 2
println!("{}", Rc::weak_count(&a));   // 1

// a Weak must be upgraded before use
if let Some(rc) = w.upgrade() {         // Some while alive
    println!("still alive: {rc}");     // 5
}                                       // None once strong = 0
```

- **Strong** (`strong_count`) - the **owning** handles from `new`/`clone`. When it hits **0**, the value is **dropped**.
- **Weak** (`weak_count`) - **non-owning** handles from `downgrade`. They **do not** keep the value alive.
- `upgrade()` turns a `Weak` into an `Option<Rc>` - `None` if the value is already gone, so you can never read freed memory.

Use `Weak` to break **reference cycles** (e.g. child ⇨ parent): a cycle of **strong** Rcs never reaches 0 and **leaks**. `Arc` works identically via `Arc::downgrade`.

Shared + mutable (single thread): `Rc<RefCell<T>>`



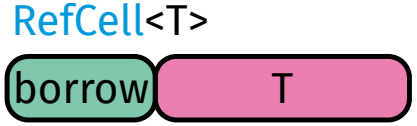
Concept:

- Rc gives **many owners** but only shared **reading**.
- Put a `RefCell` inside to also **mutate** ⇨ borrow rules then checked **at runtime**.

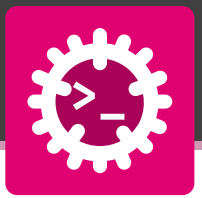
```rust
use std::rc::Rc;
use std::cell::RefCell;

fn main() {
    // one bank account, co-owned by two parts of the app
    let account = Rc::new(RefCell::new(0));
    let teller  = Rc::clone(&account);   // second owner

    *account.borrow_mut() += 100;        // deposit
    *teller.borrow_mut()  -= 30;         // withdraw

    // both handles see the same balance
    println!("balance: {}", account.borrow()); // 70
}
```

`Rc<RefCell<T>>`

- **single thread**: shared **and** mutable
- Rc - **many owners** of one value
- `RefCell` - mutate through `&`, rules checked **at runtime** (**panics** if you alias `&mut`)



Shared + mutable (multi-thread): `Arc<Mutex<T>>`



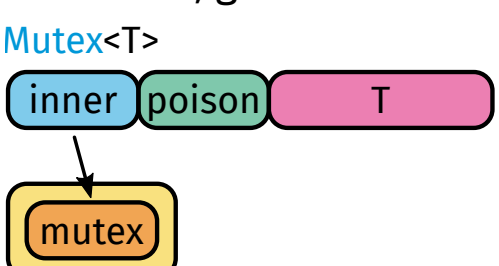
Concept: the **same shape across threads**

- `Arc` shares ownership atomically
- `Mutex` lets **one thread at a time** mutate the value safely.

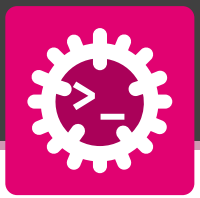
```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    // same account, now shared across worker threads
    let account = Arc::new(Mutex::new(0));

    let mut handles = vec![];
    for _ in 0..3 {
        let account = Arc::clone(&account); // owner per thread
        handles.push(thread::spawn(move || {
            *account.lock().unwrap() += 100; // safe deposit
        }));
    }
    for h in handles { h.join().unwrap(); }

    println!("balance: {}", *account.lock().unwrap()); // 300
}
```

`Arc<Mutex<T>>`

- **across threads**: shared **and** mutable
- `Arc` - **atomic** many-owners (`Send+Sync`)
- `Mutex` - `lock()` waits its turn, grants **exclusive** access, guard **auto-unlocks**



Interior mutability (single thread): `Cell<T>`



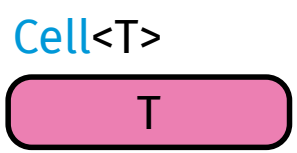
Concept: mutate a value **through a shared &**

- no `&mut`, no `Rc`, no lock. The lightweight tool for a single `Copy` field (a counter, a flag).

```rust
use std::cell::Cell;

struct Button { clicks: Cell<u32> }

impl Button {
    // &self, yet we still mutate the counter
    fn click(&self) {
        self.clicks.set(self.clicks.get() + 1);
    }
    fn count(&self) -> u32 { self.clicks.get() }
}

fn main() {
    let b = Button { clicks: Cell::new(0) };
    b.click();
    b.click();
    println!("clicks: {}", b.count()); // 2
}
```

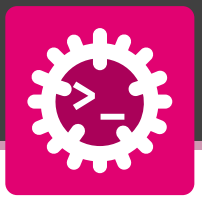
`Cell<T>`

- mutate through `&self` (no `&mut`)
- `get`/`set` **swap the whole value**
- for small **`Copy`** types, **never panics**
- **single thread only**



A `Cell` holds one **`Copy`** value.
To mutate a **non-`Copy`** value (a `Vec`, a `String`) through `&`, reach for `RefCell`.

Interior mutability (multi-thread): `Atomic*`



Concept: the **lock-free** twin of `Cell` for one `Copy` value

- a single hardware instruction does the update, safe to share across threads via `Arc`.

```rust
use std::sync::Arc;
use std::sync::atomic::{AtomicU32, Ordering};
use std::thread;

fn main() {
    // shared hit counter, no lock needed
    let hits = Arc::new(AtomicU32::new(0));

    let mut handles = vec![];
    for _ in 0..4 {
        let hits = Arc::clone(&hits);
        handles.push(thread::spawn(move || {
            for _ in 0..25 {
                hits.fetch_add(1, Ordering::Relaxed);
            }
        }));
    }
    for h in handles { h.join().unwrap(); }

    println!("hits: {}", hits.load(Ordering::Relaxed)); // 100
}
```

`Atomic*`

- `AtomicU32`, `AtomicBool`, `AtomicUsize`, ...
- **lock-free**: one atomic CPU instruction
- share across threads with `Arc`
- perfect for **counters** and **flags**

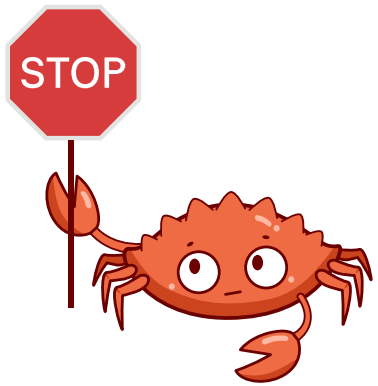
Atomics fit only **`Copy`** values. For a **non-`Copy`** shared value use `Mutex`/`RwLock`. (`Pin<P>` and `Cow<'a, T>` exist too, but are out of scope here.)

Exercise - Smart Pointers

For each situation, name the **smart pointer** you would reach for.

1. A **single owner** of a large value that must live on the **heap**.
2. **Several owners** reading the same data, all on **one** thread.
3. **Several threads** that must **share and mutate** one value.
4. **Shared and mutable** data among many owners, on **one** thread.

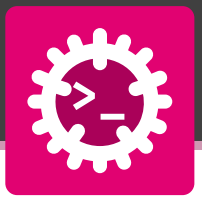
Two questions decide it: **how many threads?** and **read or write?**



Synchronization & Hazards

Locks, races and deadlocks

Critical sections & deadlock



Critical section - code that must run **exclusively**, one thread inside at a time.

- Guard it with a `Mutex<T>` (or `RwLock<T>` for read-mostly data).
- Keep it **short** - it **serialises** your program (remember Amdahl!).
- Limit to **N** concurrent users? Use a **semaphore** (`tokio::sync::Semaphore`).

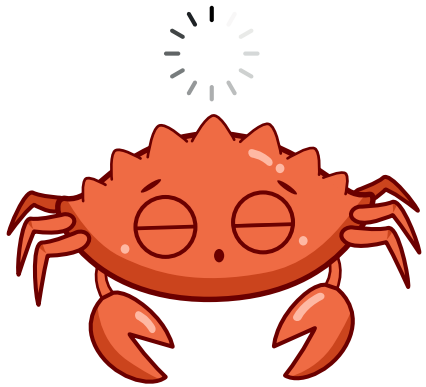
Deadlock - two threads wait **forever** on each other's locks:

- Thread A holds lock 1, wants lock 2
- Thread B holds lock 2, wants lock 1
- Neither can proceed ⇨ **frozen**.

How to avoid it

- **Lock ordering** - always acquire locks in the **same order**
- **Timeouts** - `try_lock` and back off
- **Less shared state** - prefer **message passing**

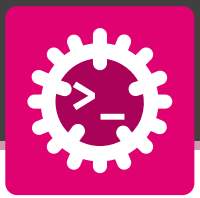
Rust stops **data races**, **not** deadlocks - those are a **design** problem.



Asynchronous Execution Model

Make progress while you wait

A different execution model



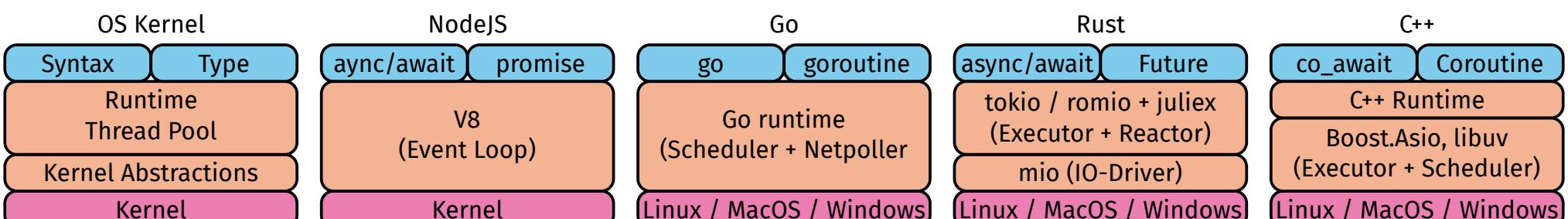
Asynchronous execution is not threads - it is a **separate model**: many **tasks** driven by a **runtime**, instead of **threads** scheduled by the OS.

Threads - the OS way

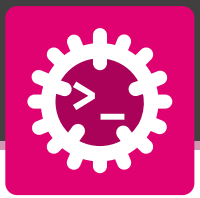
- one **stack** per task (**~MB**), **preemptively** scheduled
- **tens of thousands** of connections ⇨ as many threads
- most sit **blocked on I/O**, burning memory

Async - the runtime way

- thousands of **tasks** on a **handful** of threads
- a task is a cheap **state machine**, **cooperatively** scheduled
- ideal for **I/O-bound** waiting at scale



async / .await

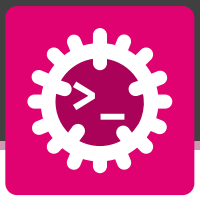


Write code that **reads** sequentially but **doesn't block** the thread.

```rust
// `async fn` returns a Future instead of running now
async fn fetch(id: u32) -> String {
    // .await yields the thread while the IO is pending
    let body = download(id).await;
    format!("got {} bytes", body.len())
}

#[tokio::main]
async fn main() {
    // both tasks share ONE thread, interleaving at .await
    let (a, b) = tokio::join!(fetch(1), fetch(2));
    println!("{a} / {b}");
}
```

- `async fn` - turns code into a **Future**
- `.await` - "**pause here** until ready, **let others run** meanwhile"
- `#[tokio::main]` - starts the **runtime** that drives it all

Cooperative scheduling

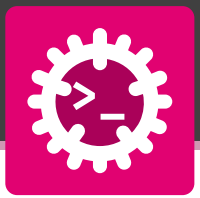


Idea: run **thousands** of tasks on a **handful** of threads - each one **yields** at `.await` instead of blocking.

```rust
async fn handler() {
    // ✕ blocks the WHOLE thread - every other
    //    task on it is frozen until this returns
    std::thread::sleep(Duration::from_secs(1));

    // ✅ yields the thread, others make progress
    tokio::time::sleep(Duration::from_secs(1)).await;
}
```

- **Preemptive** (OS threads) - the kernel can **interrupt** anytime.
- **Cooperative** (async) - a task runs until it **yields** (`.await`).
- **Never block** inside async: no `thread::sleep`, no heavy CPU loops.
- Offload CPU work to **threads / Rayon**, or use `spawn_blocking`.

Under the hood: `async fn` is a state machine

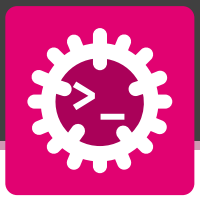


The compiler **rewrites** every `async fn` into a **state machine** - one state per `.await`.

```rust
async fn fetch(id: u32) -> String {
    let body = download(id).await; // suspension point
    format!("got {} bytes", body.len())
}

// the compiler turns `fetch` into a state machine:
enum Fetch {
    Start { id: u32 },              // before the .await
    Downloading { fut: Download }, // parked at the .await
    Done,                           // value ready
}

#[tokio::main]
async fn main() {
    // the runtime polls that machine to completion
    let s = fetch(1).await;
    println!("{s}");
}
```

- Each `.await` is a **suspension point** ⇨ a **state**
- `poll` advances the machine **one state at a time**
- Locals alive **across** an `.await` are **stored in the future**
- Nothing is blocked - the task just **remembers where it was**

Under the hood: a `Future` is lazy



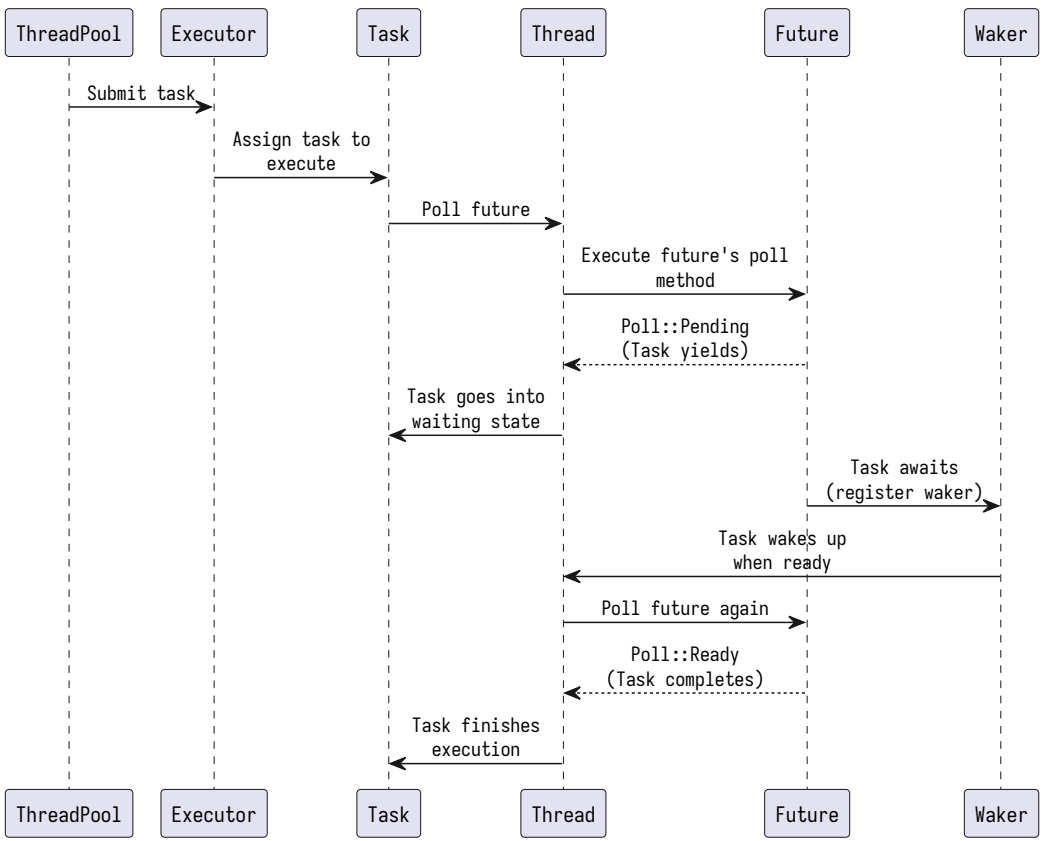
An async block **does nothing** until something **polls** it.

```
enum Poll<T> {
    Ready(T),   // done, here is the value
    Pending,    // not yet - poll me again later
}

// (simplified) a Future is polled to completion
trait Future {
    type Output;
    fn poll(&mut self, cx: &mut Context)
        -> Poll<Self::Output>;
}
```

- A `Future` is a **state machine** that is **polled**.
- `Pending` ⇨ not ready; `Ready(v)` ⇨ finished.
- **Lazy**: nothing runs until an **executor** polls it.
- So who polls, and how often? ⇨ the **runtime**.

Under the hood: the async runtime

The runtime is a handful of pieces that **poll your futures to completion**.



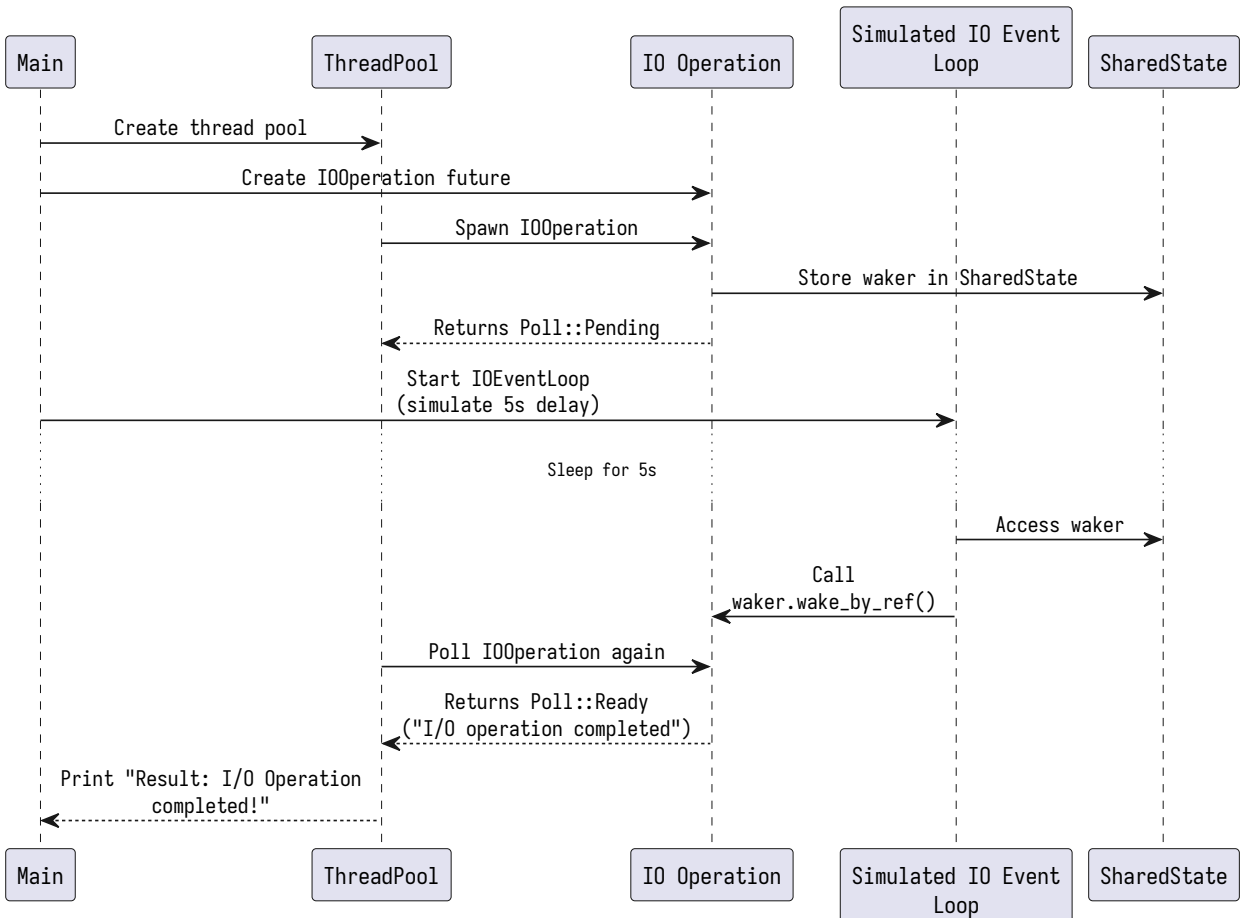
Elements of a runtime

- **Executor** - **polls** futures on a **thread pool**
- **Reactor** - watches the OS for I/O readiness
- **Waker** - the "**call me back**" handle of a parked task

The poll cycle

1. poll ⇨ **Pending**, **register a Waker**
2. task **parked** - uses **no** CPU
3. I/O ready ⇨ the **Waker** wakes it
4. poll again ⇨ **Ready** ⇨ done

Under the hood: putting it together



- A parked task **stores its waker**
- when the I/O event fires, `wake()` schedules a **re-poll** ⇨ Ready.

Threads vs. async - which one?

Threads / Rayon	Async / Tokio
best for **CPU-bound** work	best for **I/O-bound** work
true **parallelism** across cores	huge **concurrency** on few threads
heavy: MB stack per thread	cheap: a task is just a state machine
`spawn` + `join`, `par_iter`	`async`/`.await`, a **runtime**

Rule of thumb

- compute ⇨ threads
- wait (network, disk) ⇨ async.

Exercise - Async

The function should **not block** the thread while it waits. Add the **two** missing pieces and answer the question.

```
// (1) make this function asynchronous
fn fetch(id: u32) -> String {
  // (2) wait for the future without blocking the thread
  let body = download(id);
  format!("got {} bytes", body.len())
}
```

Question: why use `.await` here instead of a blocking call?

Inter-Thread Communication

Talk, don’t share

What is inter-thread communication?

Threads need to **coordinate** and **hand data** to each other. There are **two ways**.

Share memory - with Smart Pointers, Mutexes, and RwLocks

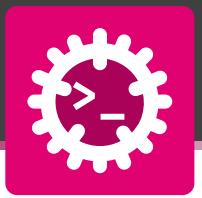
- one value behind `Arc<Mutex<T>>`
- every thread **locks**, touches it, unlocks
- risk of **races** (without the lock) and **deadlocks**

Pass messages - with channels

- threads **send owned values** down a **channel**
- the receiver **owns** what it gets - no shared lock
- sidesteps data races **and** most deadlocks

"Don't communicate by sharing memory; share memory by communicating." - the Go proverb, equally true in Rust.

Ways to pass messages



From **threads in one process** to **separate processes** - same idea, different reach.

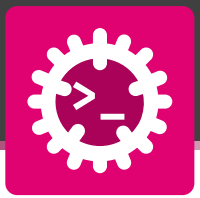
Channels - between threads, one process

Mechanism	What it is
`mpsc`	**multi-producer, single-consumer** queue - many senders, one receiver (in `std`)
`oneshot`	a **single** value sent **once** - perfect for a reply / result (Tokio)
`broadcast`	**many consumers** each receive **every** message - fan-out (Tokio)

OS-level Inter-Process Communication (IPC) - between processes

Mechanism	What it is
pipes	a **byte stream** from one process into another
sockets	a two-way channel, even **across machines** (the network)
files / shared mem	a **region** both processes read and write

The workhorse: an `mpsc` channel



The **most common** channel: many threads **send**, one thread **receives**.

```rust
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();      // sender, receiver

    thread::spawn(move || {
        tx.send("hello from worker").unwrap(); // value MOVES in
    });

    let msg = rx.recv().unwrap();        // blocks until a message
    println!("{msg}");                   // hello from worker
}
```

- `channel()` hands back a **sender** `tx` and **receiver** `rx`
- `send` **moves** the value into the channel - the sender gives it up
- `recv` **blocks** until a message arrives (or every sender is dropped)
- **clone `tx`** to get **multiple producers**

Channels turn **shared-state** problems into simple **producer / consumer** flows.

Exercise - Inter-Thread Communication

Let **3 worker threads** report their result to `main` through an `mpsc` channel - **no shared lock**.

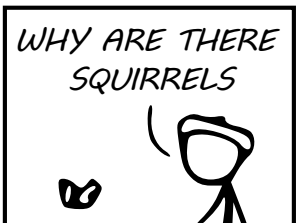
1. Create a channel with `mpsc::channel()`.
2. Spawn **3** threads; each **clones** the sender and `send`s its `id * 10`.
3. In `main`, **receive** the 3 values and print their **sum**.

Clone `tx` for multiple producers; `send` **moves** the value into the channel; `recv` **blocks** until a message arrives.

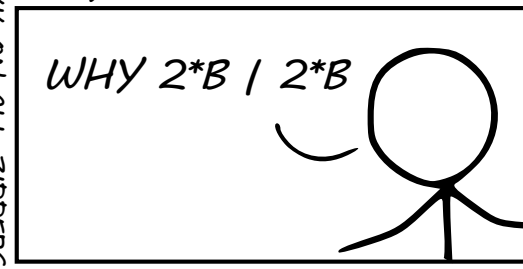
QUESTIONS

CAN BE ASKED BY ANYONE ANYTIME

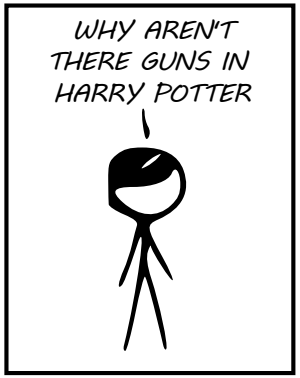
WHY DO WHALES JUMP
WHY ARE THERE MIRRORS ABOVE BEDS
WHY DO I SAY UH
WHY IS SEA SALT BETTER
WHY ARE THERE TREES IN THE MIDDLE OF FIELDS
WHY IS THERE NOT A POKEMON MMO
WHY IS THERE LAUGHING IN TV SHOWS
WHY ARE THERE DOORS ON THE FREEWAY
WHY ARE THERE SO MANY SVCHOST·EXE RUNNING
WHY AREN'T ANY COUNTRIES IN ANTARCTICA
WHY ARE THERE SCARY SOUNDS IN MINECRAFT
WHY IS THERE KICKING IN MY STOMACH
WHY ARE THERE TWO SLASHES AFTER HTTP
WHY ARE THERE CELEBRITIES
WHY DO SNAKES EXIST
WHY DO OYSTERS HAVE PEARLS
WHY ARE DUCKS CALLED DUCKS
WHY DO THEY CALL IT THE CLAP
WHY ARE KYLE AND CARTMAN FRIENDS
WHY IS THERE AN ARROW ON AANG'S HEAD
WHY ARE TEXT MESSAGES BLUE
WHY ARE THERE MUSTACHES ON CLOTHES
WHY WUBA LUBBA DUB DUB MEANING
WHY IS THERE A WHALE AND A POT FALLING
WHY ARE THERE SO MANY BIRDS IN SWISS
WHY IS THERE SO LITTLE RAIN IN WALLIS
WHY IS WALLIS WEATHER FORECAST ALWAYS WRONG
WHY ARE THERE MALE AND FEMALE BIKES
WHY ARE THERE BRIDESMAIDS
WHY DO DYING PEOPLE REACH UP
HOW FAST IS LIGHTSPEED
WHY ARE OLD KLINGONS DIFFERENT
WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPYDER BITES ITCH



WHY ARE HATS SO EXPENSIVE
WHY IS THERE CAFFEINE IN MY SHAMPOO
WHY HAVE DINOSAURS NO FUR
WHY DO TWINS HAVE DIFFERENT FINGERPRINTS
WHY ARE SWISS AFRAID OF DRAGONS
WHY IS HTTPS CROSSED OUT
WHY IS THERE A LINE THROUGH
WHY IS THERE A RED LINE THROUGH HTTPS
WHY IS HTTPS IMPOR

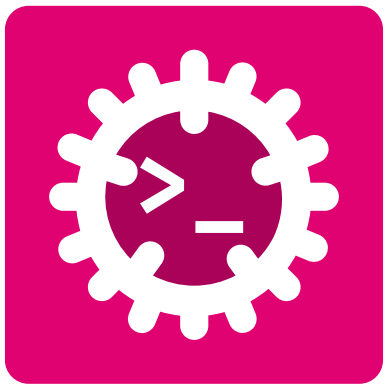


WHY AREN'T ECONOMISTS RICH
WHY DO AMERICANS CALL IT SOCCER
WHY ARE MY EARS RINGING
WHY IS 42 THE ANSWER TO EVERYTHING
WHY CAN'T NOBODY ELSE LIFT THORS HAMMER
WHY IS MARVIN ALWAYS SO SAD
WHY ARE THERE ANTS IN MY LAPTOP
WHY IS EARTH TILTED
WHY IS SPACE BLACK
WHY IS OUTER SPACE SO COLD
WHY ARE THERE PYRAMIDS ON THE MOON
WHY IS NASA SHUTTING DOWN
WHY IS THERE A SWARM OF ANTS
WHY IS THERE PILEGRIM
WHY ARE THERE SO MANY CROWS IN ROCHES
WHY IS TO BE OR NOT TO BE FU
WHY DO CHILDREN GET CANCER
WHY IS POSEIDON ANGRY WITH ODYSSEUS
WHY IS THERE ICE IN SPACE

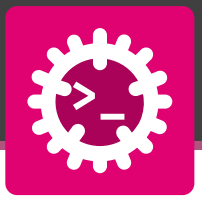


WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE FPGA's EVERYWHERE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOCKS
WHAT IS https://xkcd.com/1256/
WHY DO THEY SAY T-MINUS
WHY ARE OCEANS BECOMMING MORE ACIDIC
WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO PEOPLE DIE

WHY DO IGUANAS DIE
WHY AREN'T THERE DINOSAUR GHOSTS
WHY IS THERE LAVA
WHY IS THERE HE
WHY ARE THERE FEMA
WHY IS YKK ON ALL ZIPPERS



Glossary & Bibliography



Computer Science

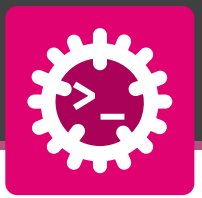
CPU – **Central Processing Unit**: The main processing component of a computer. 20, 20, 20, 21, 21, 21, 21, 21, 44, 44, 47, 49

OS – **Operating System**: System software that manages hardware resources and applications. 1, 7, 10, 19, 20, 20, 20, 42, 42, 44, 47, 53

Concurrency

I/O – **Input/Output**: Operations that involve reading from or writing to external devices or resources. 20, 20, 20, 21, 21, 42, 42, 47, 47, 48, 49

IPC – **Inter-Process Communication**: Mechanisms allowing processes to exchange data and synchronize execution. 53



Bibliography

[1] S. Klabnik, C. Nichols, and C. Krycho, “The Rust Programming Language.”

[2] J. Blandy, J. Orendorff, and L. F. s Tindall, *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, Incorporated, 2021.

[3] H. Collard, *Black Hat Rust*, vol. 1. Amazon, 2025.

[4] A. Shvets, “Refactoring Guru.” 2026.

[5] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education, 2008.

[6] “SVG Repo - Free SVG Vectors and Icons.”